Mercatino Libri Usati
Versione Digitale

RASD + DD + ITD

Jaunary 22, 2025
Version 3.0

Author: Elia Pontiggia
Mediator: Filippo Ambrosini

Revision History

Date	Revision	Notes
22/01/2025	v.0.0	Document creation
23/01/2025	v.1.0	First release
31/01/2025	v.2.0	Deployment of the first version of the system
20/06/2025	v.3.0	Updates and improvements, deployment to the public use. API e

Contents

1. Introduction

1.1 Purpose

The purpose of the MLUD is to digitize and streamline the process of cataloging and monitoring the used schoolbook market held at Lokalino during the summer. The system aims to eeduce the workload on operators, particularly for repetitive tasks, minimize errors by eliminating manual transcription processes and significantly decrease paper usage.

1.2 Scope

The MLUD will be used by operators, providers and buyers. The system will allow operators to manage the catalog of books, providers to add books to the catalog and buyers to search for books. The system will also allow operators to account for the books sold and help the financial transactions between providers and buyers. The system does not handle actual financial transactions but keeps a record of the money involved in the transactions.
The boundaries of the system are as follows:

- Operations are limited to the Lokalino summer book market.
- The platform will only retain data for one year after market closure, except for users who opt into the mailing list.

1.3 Definitions, acronyms and abbreviations

Term	**Definition**	**Acronym**
Operator	A Lokalino employee	OP
Provider	A person who has books to sell	PR
Buyer	A person who wants to buy books	BY
Mercatino Libri	The system	MLUD

2. Requirement Analysis and Specification

The MLUD will offer the following functionalities:

- User registration and book detail submission via an online form.
- Facilitating the physical handover of books, including verification and labeling.
- Streamlining the sales process with a digital interface for managing transactions.
- Tracking unsold books and managing financial settlements for sellers.
- Generating aggregated statistical reports to monitor performance and user satisfaction.

2.1 Hardware Interfaces

The system will be accessible through a web interface. The PR will have the option to use both a computer and a mobile device to access the system, so the interface must be responsive and usable on both platforms.

The OP will use a computer to access the system, so the interface must be optimized for computer use.

2.2 Use Cases

The use cases are illustrated in the diagram in Figure 2.1.

2.2.1 Use Case 1: Book submission

1. The PR accesses the system.
2. The PR fills out the form, providing:
    - Personal information (name, surname, email, phone number, school of origin).

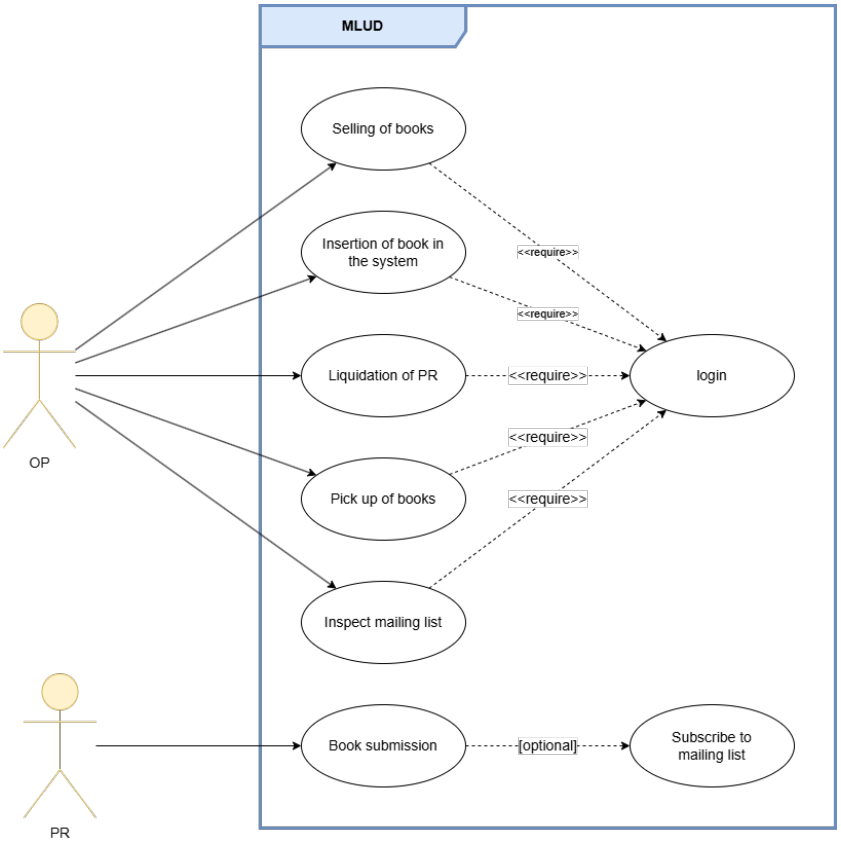

Figure 2.1: Use cases diagram

- Information for an arbitrary number of books (ISBN, title, author, edition, price, condition).
- Acceptance of the terms and conditions.
- Acceptance of Lokalino rules.
- Preference for subscription to the mailing list.
- Approximate period of delivery of the books.

When a period of delivery is selected, the system will show the number od PR that have already submitted the form for that period, so that the PR can choose a period with a reasonable number of PRs.
When entering the ISBN or the title of the book, the system will suggest books that are already in the system (that are books that have been adopted by the schools in the area), so that the PR can select them and avoid entering the same book multiple times. If the book is not in the system, the PR cannot enter it manually, because most likely it is not adopted anymore, but the PR can still bring it to Lokalino and the OP will be able to verify it and add it to the system if it is correct.

3. The PR submits the form, it is redirected to a confirmation page.

2.2.2 Use Case 2: Book delivery

1. The PR goes to Lokalino with the books.
2. The OP accesses the protected area of the system.
3. The OP searches for the record of the PR in the system.
4. The OP verifies the correspondence between the books and the records, eventually updating the records or adding comments (To help the OP to keep track of verified books, each book of the list will present a checkbox which will have no practical impact, just visual, as shown in figure 3.9)
5. If all is correct, the OP labels the books to identify the corresponding PR
6. The OP mark the correct delivery of the books in the system

2.2.3 Use Case 3: Book sale

1. The OP accesses the protected area of the system.
2. The BY selects the books they want to buy.
3. For each book, the OP searches for the record in the system and adds the book to the cart.
4. At the end of the selection, the OP confirms the purchase and collects the payment. The books are marked as sold in the system.

2.2.4 Use Case 4: Liquidation and return of unsold books

1. The OP accesses the protected area of the system.
2. The OP searches for the record of the PR in the system.
3. The system shows the list of books unsold and the total amount due to the PR.
4. The PR collects the money and the unsold books.
5. The OP marks the books as returned in the system.

2.3 Special Requirements

- In order to generate the list of books that can be suggested to the PR, the system will provide the possibility to the OP to insert the list of books that have been adopted by the schools in the area. The OP will be able to insert the list of books in a facilitated way, to be defined directly with the OP a short time before the delivery.
- The system will keep the data of the PRs and the books for 1 year after the event, and after that, it will be deleted. The only data that will be kept indefinitely is the list of persons who have subscribed to the mailing list.
- Every time an OP handles information about a PR, the system will also show the unique ID of the PR, so that the OP can easily label and identify the books.
- Since the prices of the books are different from year to year, the system will not compile automatically the price of the books when the PR fills out the form, as it does for the Author and Editor. The PR will have to fill out the price of the book manually, and the OP will have to verify it when the books are delivered. If the price is not correct, the OP will have to delete the book from the system and re-enter it with the correct price.

2.4 Assumptions

An assumptions have been made in the design of the system:

Even if in the implementation every PR will be identified by a unique ID, the system will check the uniqueness of the email address; this comes into play when a PR submits the form: if the email address is already in the system and the PR fills out the form again, the system will not create a new record and will work as if the same PR is inserting more books.

3. Design

3.1 Component diagram

The system will be organized in a classical client-server configuration, and the latter will expose RESTful APIs to the clients, and a web application for both OPs and PRs. Every service will be in its own servlet. The component diagram is shown in Figure 3.1.

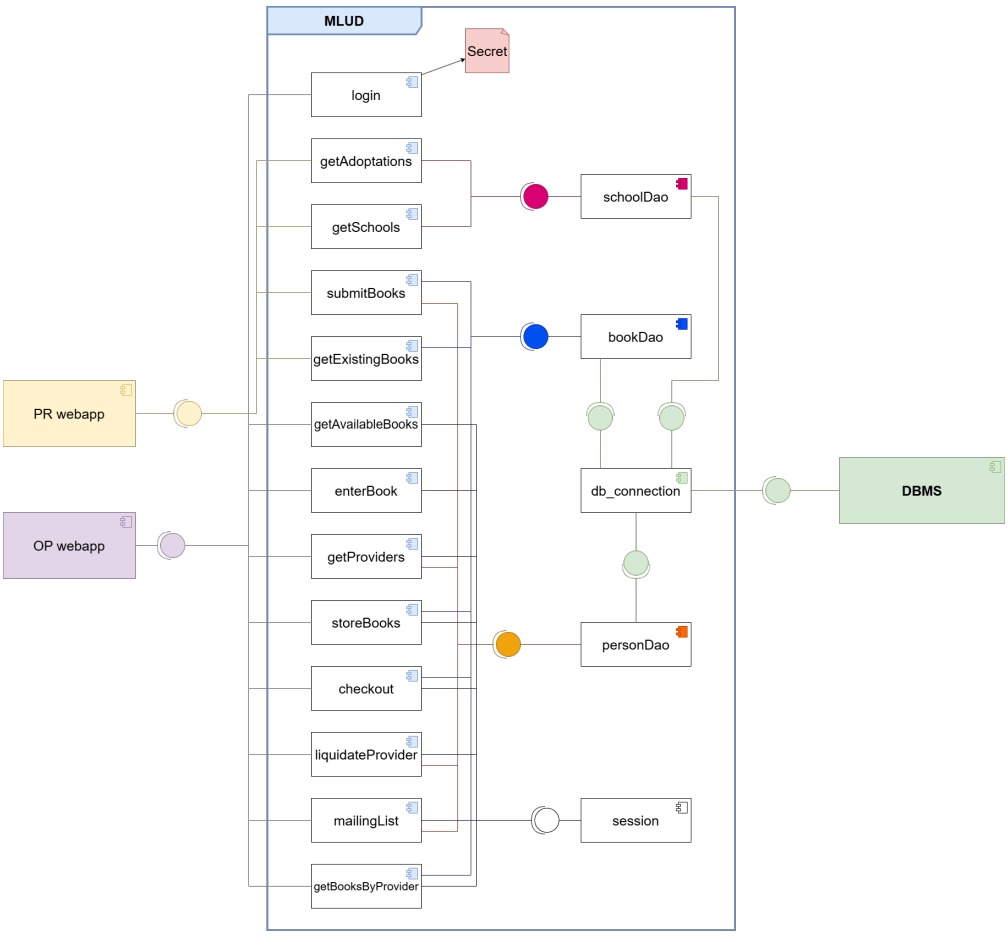

Figure 3.1: Component diagram

3.2 Logical description of data

The data will be stored into a relational database, which will be designed according to the Logical schema in Figure 3.2.

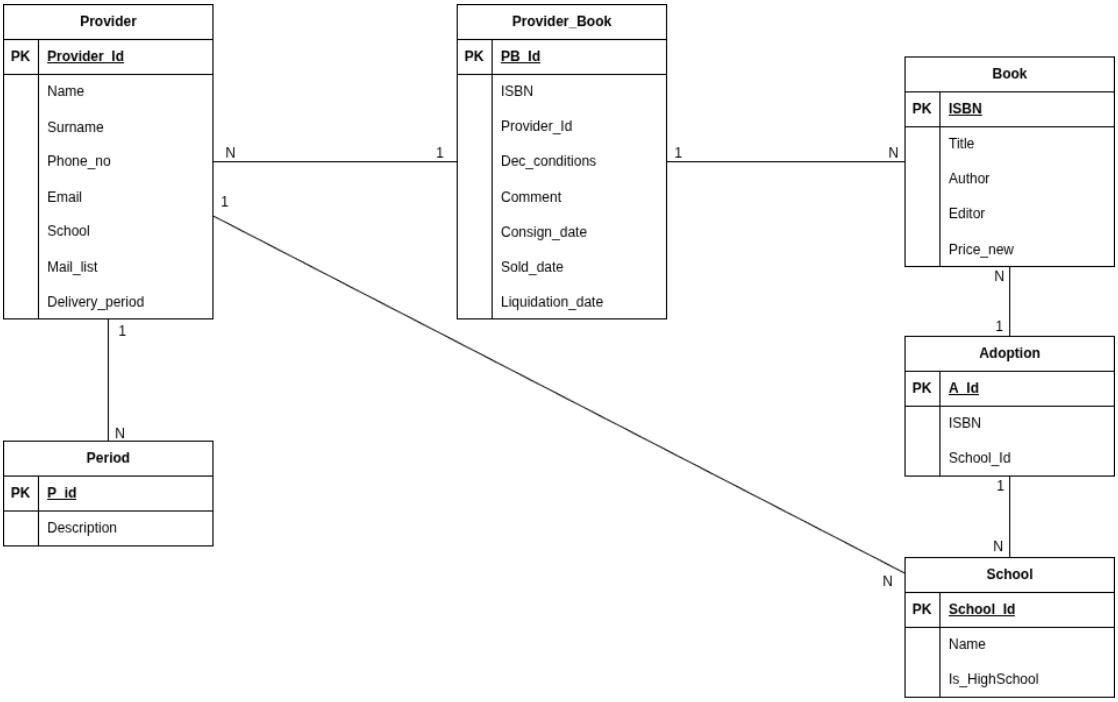

Figure 3.2: Logical schema of the database

3.3 API endpoints

The API exposes endpoints in a structure coherent with the component diagram 3.1. The following sections describe the available endpoints.

3.3.1 Login

Method: POST
EndPoint: /login.php
Body parameters:

```
{ "Password": String }
```

Response: `200` if the login is successful, `401` if the password is wrong.

3.3.2 Submit Books

A PR submits its books to the system together with personal information.

Method: POST
EndPoint: /submitBooks.php
Body parameters:

```
{
    "personalInfo": {
        "Name": String,
        "Surname": String,
        "School": String,
        "Email": String,
        "Phone_no": String,
        "Mail_list": Boolean
    },
    "books": [
        {
            "ISBN": String,
            "Title": String,
            "Author": String,
            "Editor": String,
            "Price_new": Number,
            "Dec_conditions": String
        }
    ]
}
```

Response: `200` if the submission is successful, `400` if the request is malformed.

3.3.3 Get Existing Books

Returns information about books already present in the system, searched by ISBN or Title.

Method: GET
EndPoint: /getExistingBooks.php
URL parameters:

- `ISBN`: String
- `Title`: String

Response:

```
[
    {
        "ISBN": String,
        "Title": String,
        "Author": String,
        "Editor": String,
        "Price_new": Number
    }
]
```

3.3.4 Get Books in Stock

Returns all books currently in stock, i.e. books submitted and physically delivered by PRs.

Method: GET
EndPoint: /getAvailableBooks.php

Response:

```
[
    {
        "ISBN": String,
        "Title": String,
        "Author": String,
        "Editor": String,
        "Price_new": Number,
        "ProviderName": String,
        "ProviderSurname": String,
        "PB_Id": Number,
        "Provider_Id": Number,
        "Dec_conditions": String,
        "Comment": String,
        "Consign_date": String,
        "Sold_date": null,
        "Liquidation_date": null
    }
]
```

3.3.5 Enter Book

An OP manually inserts a new book in the system to simplify PR submissions.

Method: POST
EndPoint: /enterBook.php
Body parameters:

```
{
    "ISBN": String,
    "Title": String,
    "Author": String,
    "Editor": String,
    "Price_new": Number
}
```

Response: `200` if successful, `400` if malformed.

3.3.6 Get PRs

Returns the list of PRs.

Method: GET
EndPoint: /getProviders.php

Response:

```
[
    {
        "Provider_Id": Number,
        "Name": String,
        "Surname": String,
        "State": String
    }
]
```

Possible states:

- `0`: waiting for delivery
- `1`: waiting for liquidation
- `2`: liquidated

3.3.7 Get Books Submitted by a PR

Returns all books submitted by a specific PR.

Method: GET
EndPoint: /getBooksByProvider.php

URL parameters:

- `Provider_Id`: Number

Response:

```
{
    "books": [
        {
            "PB_Id": Number,
            "ISBN": String,
            "Title": String,
            "Author": String,
            "Editor": String,
            "Price_new": Number,
            "Dec_conditions": String,
            "Sold_date": Timestamp
        }
    ],
    "provider": [
        {
            "Name": String,
            "Surname": String
        }
    ]
}
```

3.3.8 Deliver Books

A PR delivers books to the OP.

Method: POST
EndPoint: /storeBooks.php

Body parameters:

```
{
    "Provider_Id": Number,
    "Books_to_edit": [
        {
            "PB_Id": Number,
            "Dec_conditions": String,
            "Comment": String
        }
    ],
    "Books_to_add": [
        {
```

```
            "ISBN": String,
            "Title": String,
            "Author": String,
            "Editor": String,
            "Price_new": Number,
            "Dec_conditions": String,
            "Comment": String
        }
    ],
    "Books_to_remove": [
        {
            "PB_Id": Number
        }
    ]
}
```

Response: `200` if successful, `400` if malformed.

3.3.9 Sell Books

An OP sells books to a buyer.

Method: POST
EndPoint: /checkout.php

Body parameters:

```
[
    { "PB_Id": Number }
]
```

Response: `200` if successful, `400` if malformed.

3.3.10 Liquidate PR and Return Unsold Books

An OP liquidates a PR by returning unsold books and issuing payment.

Method: POST
EndPoint: /liquidateProvider.php

Body parameters:

```
{ "Provider_Id": Number }
```

Response: `200` if successful, `400` if malformed.

3.3.11 Mailing List

Returns the list of PRs subscribed to the mailing list.

Method: GET
EndPoint: /mailingList.php

Response:

```
[
    {
        "Name": String,
        "Surname": String,
        "Phone_no": String,
        "Email": String,
        "School": String,
        "Mail_list": Boolean
    }
]
```

3.3.12 Session Check

Checks whether the current session is still valid.

Method: GET
EndPoint: /utils/session.php

Response: `200` if valid, `401` if expired.

3.3.13 Get Schools

Returns the list of schools available in the system.

Method: GET
EndPoint: /getSchools.php

Response:

```
[
    {
        "School_Id": Number,
        "Name": String,
        "Is_HighSchool": Boolean
    }
]
```

3.3.14 Get Books Adopted by a School

Returns books adopted by a specific school.

Method: GET
EndPoint: /getAdoptedBooks.php

URL parameters:

- `School_Id`: Number

Response:

```
[
    {
        "ISBN": String,
        "Title": String,
        "Author": String,
        "Editor": String,
        "Price_new": Number
    }
]
```

3.3.15 Get Delivery Periods and Affluence

Returns delivery periods and the number of PRs associated with each period.

Method: GET
EndPoint: /getPeriodsAndAffluences.php

Response:

```
[
    {
        "P_Id": Number,
        "Description": String,
        "Num_Providers": Number
    }
]
```

3.3.16 Get Emails Grouped by Period

Returns mailing-list emails grouped by delivery period.

Method: GET
EndPoint: /getPersonsByPeriod.php

Response:

```
[
    {
        "period_Description": String,
        "emails": [
            String
        ]
    }
]
```

3.4 Stats endpoints

The following endpoints provide aggregated statistics for operators. All endpoints are accessed via **GET** under the `/stats/` path and require a valid session.

3.4.1 Get Books per School

Method: GET
EndPoint: /stats/getBooksPerSchool.php

Response:

```
[
    {
        "Name": String,
        "books_cnt": Number
    }
]
```

3.4.2 Get High-School Ratio

Method: GET
EndPoint: /stats/getHighSchoolRatio.php

Response:

```
[
    {
        "Is_HighSchool": Number,
        "persons_cnt": Number
    }
```

```
]
```

3.4.3 Get Number of New Mailing Subscribers

Method: GET
EndPoint: /stats/getNumNewMailSubscribers.php

Response:

```
123
```

3.4.4 Get Sales per Day

Method: GET
EndPoint: /stats/getSalesPerDay.php

Response:

```
[
    {
        "date_sale": String,
        "day_money": Number,
        "num_buyers": Number
    }
]
```

3.4.5 Get Total Money Movement

Method: GET
EndPoint: /stats/getTotalMoneyMovement.php

Response:

```
12345.67
```

3.5 Deployment

All the webapp will be entirely deployed on the free hosting service Altervista, which provides a MySQL database and a php server, so all the problems of deployment, reliability, scalability, and security are handled by the service provider.

Particularly, the link to the webapp will be `https://lokalinomlud.altervista.org/`

3.6 UX design

The webapp will be designed with a simple and intuitive interface, with a clean and modern look.

The UX is explained at an high level in the flow diagrams in Figures 3.3 and 3.4.

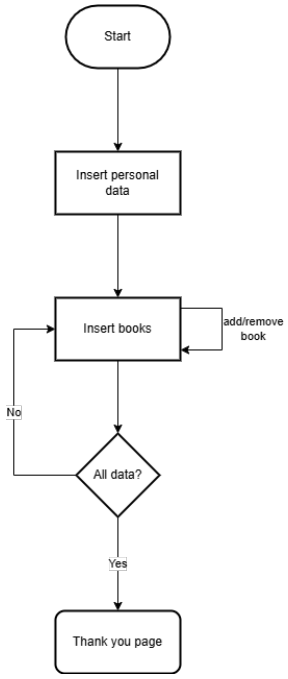

Figure 3.3: Flow of the book submission

3.7 User interface

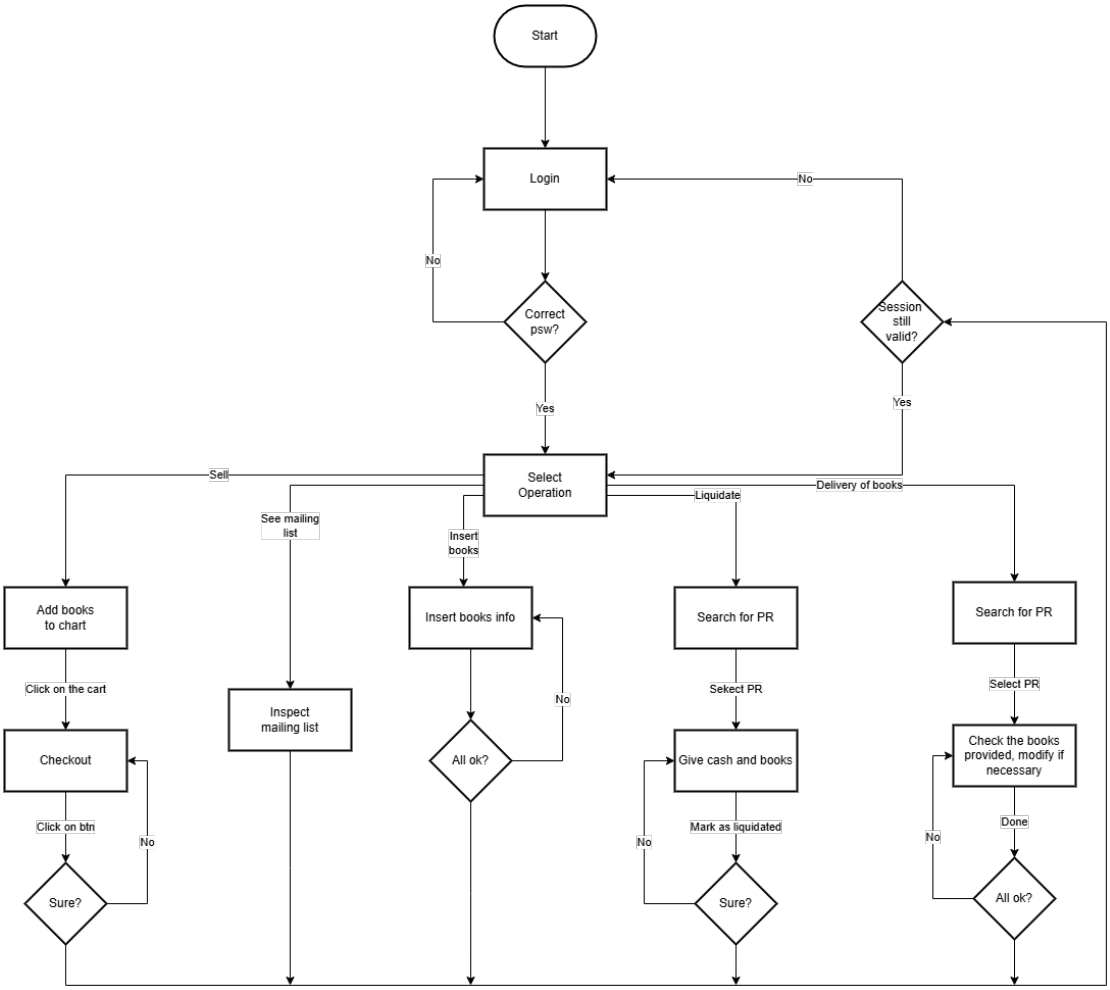

Figure 3.4: Flow of the OP

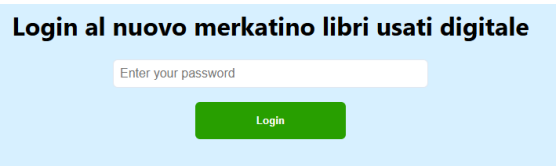

Figure 3.5: Login page

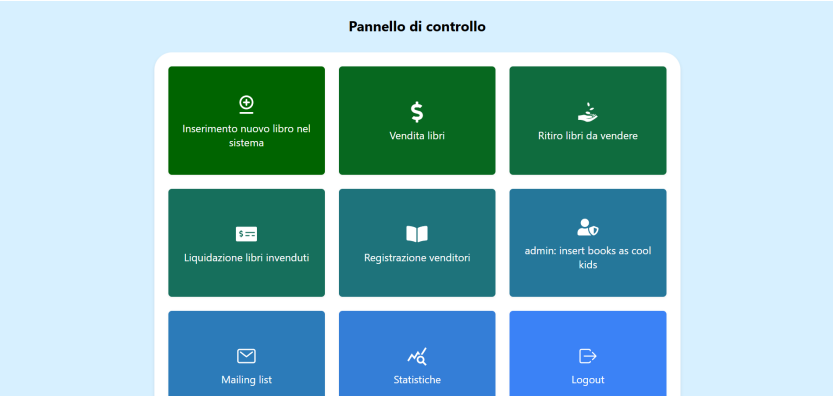

Figure 3.6: Dashboard

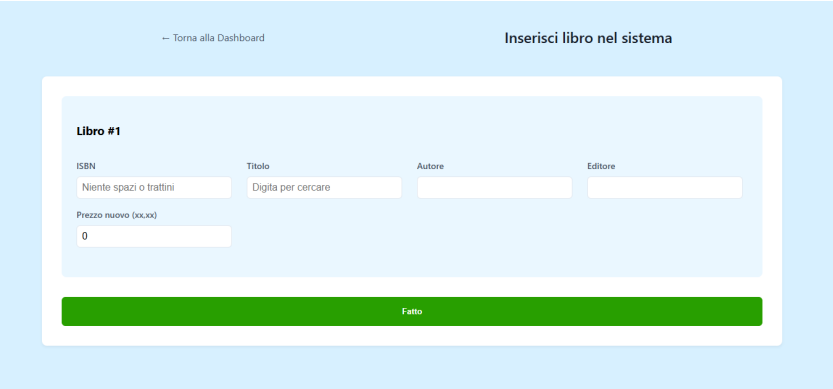

Figure 3.7: Insert book into the system

Figure 3.8: Select a PR to pick up its books

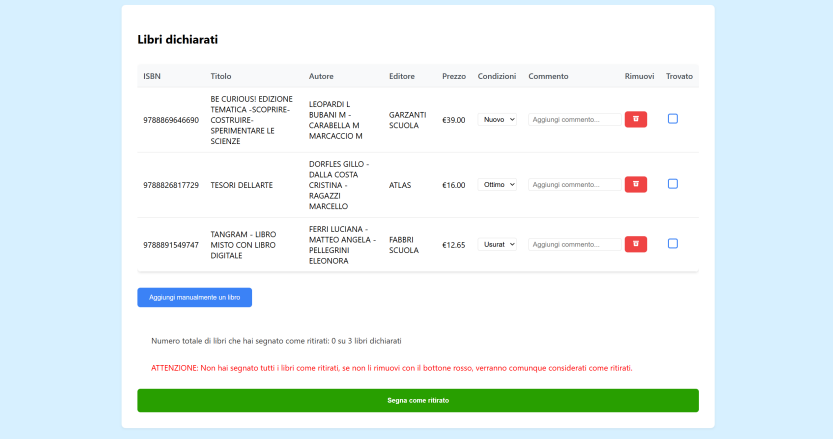

Figure 3.9: List of books to pick up from a PR

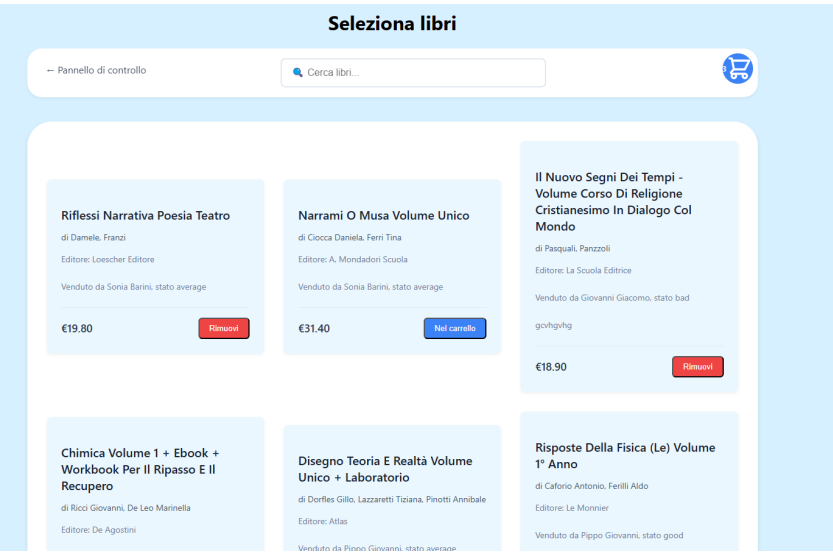

Figure 3.10: Books to sell

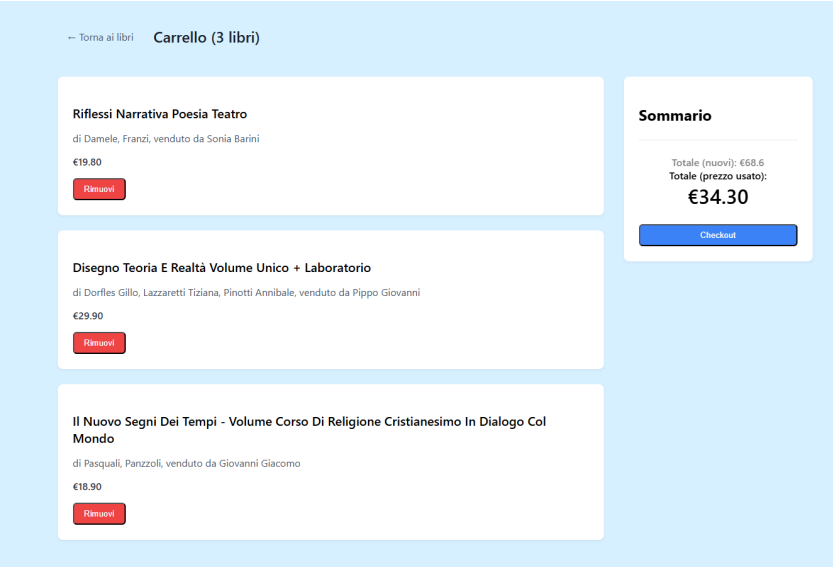

Figure 3.11: Cart

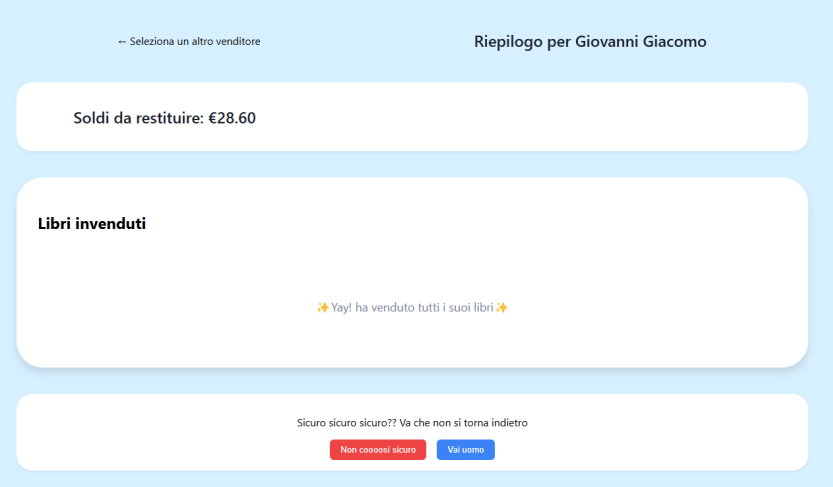

Figure 3.12: Liquidation of a PR

Figure 3.13: Mailing list

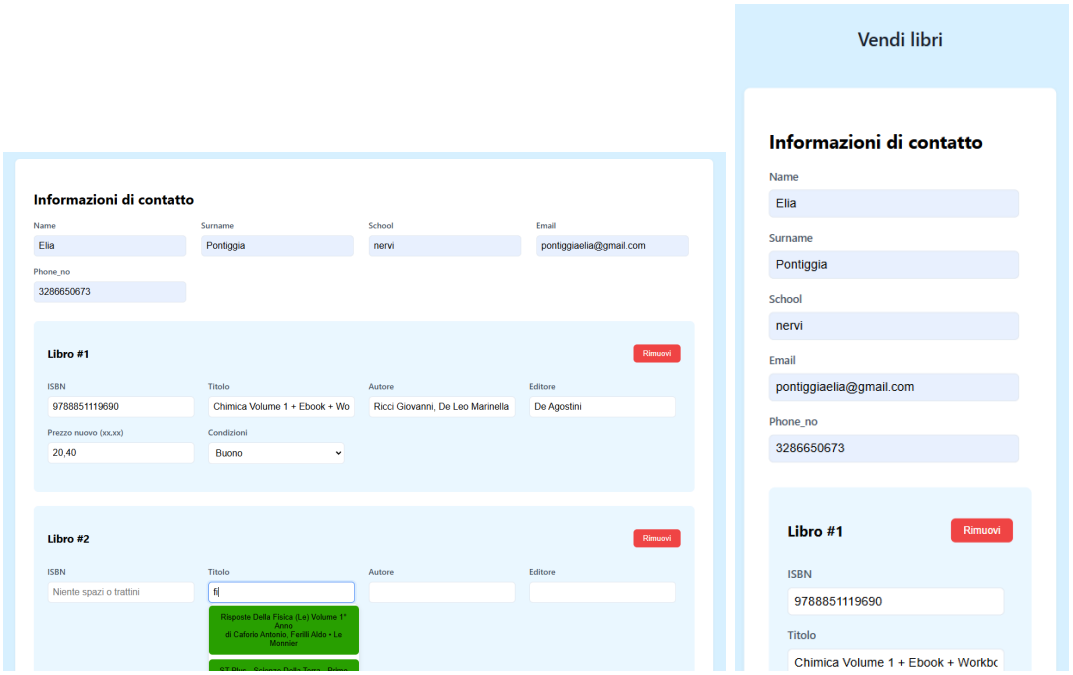

Figure 3.14: PR registration in both scenarios: from a PC and from a smartphone

4. Implementation

All the code is open source under the Apache 2.0 license and can be found on GitHub at

`https://github.com/pontig/lokalino_mlud`.

4.1 Adopted development framework

Backend

The backend is implemented in **php**, since it is the language that the author is most familiar with and it is the only backend supported by Altervista (the free hosting service used for the project).

Frontend

The frontend is implemented in **React**. React is a JavaScript library for building user interfaces. It is maintained by Facebook and a community of individual developers and companies. React can be used as a base in the development of single-page or mobile applications, since it is capable of handling the view layer for web and mobile apps. React was chosen because it is a modern and widely used library that allows for the creation of a dynamic and responsive user interface.

4.2 Structure of the source code

The source code is structured in a way that separates the frontend from the backend, both organized in an intuitive way and easy to scale and maintain.

The full structure of the source code is as follows:

`backend/` - contains the backend code

- `dao/` - contains the data access objects, one for the books and one for the PR. it also contains the database connection
- `utils/` - contains the utility functions, such as the session management and the secret passwords of the OPs
- `*.php` - the servlets that handle the requests from the frontend

`frontend/` - contains the React application

- `build/` - contains the compiled code (not included in the repository)
- `public/` - contains the static files, such as the index.html and the icons
- `src/` - contains the source code
  - `components/` - contains the reusable components
  - `pages/` - contains the pages of the application
  - `styles/` - contains the stylesheets
  - `types/` - contains the typescript interfaces
  - `App.js` - the main component of the application
  - `index.js` - the entry point of the application

4.3 Test plan

The testing of the application will be done once the development is complete. The testing will be done manually, by the OPs, simulating a real user experience.

All this process will be supervised by the author of the project.

Appendix

A. Database Creation Script

Listing A.1: Database Creation Script

```
-- phpMyAdmin SQL Dump
-- version 5.2.0
-- https://www.phpmyadmin.net/
--
-- Host: localhost
-- Creato il: Giu 18, 2025 alle 19:17
-- Versione del server: 8.0.36
-- Versione PHP: 8.0.22

SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
START TRANSACTION;
SET time_zone = "+00:00";


/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;

--
-- Database: `my_lokalinomlud`
--

-- --------------------------------------------------------

--
-- Struttura della tabella `Adoptation`
--

CREATE TABLE `Adoptation` (
  `A_Id` bigint UNSIGNED NOT NULL,
  `ISBN` char(13) NOT NULL,
  `School_Id` varchar(100) NOT NULL
) ENGINE=MyISAM DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

```
-- --------------------------------------------------------

--
-- Struttura della tabella ‘Book‘
--

CREATE TABLE ‘Book‘ (
  ‘ISBN‘ char(13) NOT NULL,
  ‘Title‘ varchar(200) NOT NULL,
  ‘Author‘ varchar(100) NOT NULL,
  ‘Editor‘ varchar(100) NOT NULL,
  ‘Price_new‘ decimal(10,2) NOT NULL
) ENGINE=MyISAM DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- --------------------------------------------------------

--
-- Struttura della tabella ‘Period‘
--

CREATE TABLE ‘Period‘ (
  ‘P_Id‘ bigint UNSIGNED NOT NULL,
  ‘Description‘ varchar(100) NOT NULL
) ENGINE=MyISAM DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- --------------------------------------------------------

--
-- Struttura della tabella ‘Provider‘
--

CREATE TABLE ‘Provider‘ (
  ‘Provider_Id‘ bigint UNSIGNED NOT NULL,
  ‘Name‘ varchar(100) NOT NULL,
  ‘Surname‘ varchar(100) NOT NULL,
  ‘Phone_no‘ varchar(15) NOT NULL,
  ‘Email‘ varchar(100) NOT NULL,
  ‘School‘ int NOT NULL,
  ‘Mail_list‘ tinyint(1) NOT NULL,
  ‘Delivery_period‘ int NOT NULL
) ENGINE=MyISAM DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- --------------------------------------------------------

--
-- Struttura della tabella ‘Provider_Book‘
--

CREATE TABLE ‘Provider_Book‘ (
```

```
  `PB_Id` bigint UNSIGNED NOT NULL,
  `ISBN` char(13) NOT NULL,
  `Provider_Id` int NOT NULL,
  `Dec_conditions` varchar(100) NOT NULL,
  `Comment` varchar(200) DEFAULT NULL,
  `Consign_date` timestamp NULL DEFAULT NULL,
  `Sold_date` timestamp NULL DEFAULT NULL,
  `Liquidation_date` timestamp NULL DEFAULT NULL
) ENGINE=MyISAM DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- --------------------------------------------------------

--
-- Struttura della tabella `School`
--

CREATE TABLE `School` (
  `School_Id` bigint UNSIGNED NOT NULL,
  `Name` varchar(200) NOT NULL,
  `Is_HighSchool` tinyint(1) NOT NULL
) ENGINE=MyISAM DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

--
-- Indici per le tabelle scaricate
--

--
-- Indici per le tabelle `Adoptation`
--
ALTER TABLE `Adoptation`
  ADD PRIMARY KEY (`A_Id`),
  ADD UNIQUE KEY `A_Id` (`A_Id`),
  ADD KEY `ISBN` (`ISBN`),
  ADD KEY `School_Id` (`School_Id`);

--
-- Indici per le tabelle `Book`
--
ALTER TABLE `Book`
  ADD PRIMARY KEY (`ISBN`);

--
-- Indici per le tabelle `Period`
--
ALTER TABLE `Period`
  ADD PRIMARY KEY (`P_Id`),
  ADD UNIQUE KEY `P_Id` (`P_Id`);

--
```

```
-- Indici per le tabelle ‘Provider‘
--
ALTER TABLE ‘Provider‘
  ADD PRIMARY KEY (‘Provider_Id‘),
  ADD UNIQUE KEY ‘Provider_Id‘ (‘Provider_Id‘),
  ADD KEY ‘School‘ (‘School‘),
  ADD KEY ‘Delivery_period‘ (‘Delivery_period‘);

--
-- Indici per le tabelle ‘Provider_Book‘
--
ALTER TABLE ‘Provider_Book‘
  ADD PRIMARY KEY (‘PB_Id‘),
  ADD UNIQUE KEY ‘PB_Id‘ (‘PB_Id‘),
  ADD KEY ‘ISBN‘ (‘ISBN‘),
  ADD KEY ‘Provider_Id‘ (‘Provider_Id‘);

--
-- Indici per le tabelle ‘School‘
--
ALTER TABLE ‘School‘
  ADD PRIMARY KEY (‘School_Id‘),
  ADD UNIQUE KEY ‘School_Id‘ (‘School_Id‘);

--
-- AUTO_INCREMENT per le tabelle scaricate
--


--
-- AUTO_INCREMENT per la tabella ‘Adoptation‘
--
ALTER TABLE ‘Adoptation‘
  MODIFY ‘A_Id‘ bigint UNSIGNED NOT NULL AUTO_INCREMENT;

--
-- AUTO_INCREMENT per la tabella ‘Period‘
--
ALTER TABLE ‘Period‘
  MODIFY ‘P_Id‘ bigint UNSIGNED NOT NULL AUTO_INCREMENT;

--
-- AUTO_INCREMENT per la tabella ‘Provider‘
--
ALTER TABLE ‘Provider‘
  MODIFY ‘Provider_Id‘ bigint UNSIGNED NOT NULL AUTO_INCREMENT;

--
-- AUTO_INCREMENT per la tabella ‘Provider_Book‘
--
```

```
ALTER TABLE `Provider_Book`
  MODIFY `PB_Id` bigint UNSIGNED NOT NULL AUTO_INCREMENT;

--
-- AUTO_INCREMENT per la tabella `School`
--
ALTER TABLE `School`
  MODIFY `School_Id` bigint UNSIGNED NOT NULL AUTO_INCREMENT;
COMMIT;

/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
/*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;
/*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;
```

B. Changelog

B.1 Version 1.0 to 2.0

- Added changelog
- Typo fixes
- Correction of the API endpoints, to match the implementation
- Correction of diagrams 2.1, 3.1, 3.4
- Added user interfaces (section 3.7)
- Added domain assumptions (section 2.4)

B.2 Version 2.0 to 3.0

- Added the "school of origin" dynamic, with all the related changes in the API endpoints and the databse structure
- Added the "price different from year to year" dynamic, with all the related changes in the API endpoints and the databse structure
- Updated the user interface to follow the client's requests
- Added the list of emails grouped by period, with both backend and frontend and api changes